

Конспект лекций по машинному обучению и анализу данных

Лекция 1 (09.02): Разведочный анализ данных (Exploratory Data Analysis, EDA)

Разведочный анализ данных (EDA) – это обязательный и фундаментальный этап разработки любой модели машинного обучения. Он направлен на глубокое понимание структуры датасета, выявление скрытых взаимосвязей, детекцию аномалий и первичную проверку рабочих гипотез. Само понятие и философия EDA были введены выдающимся математиком и статистиком **Джоном Тьюки** (John Tukey), который подчеркивал, что анализ данных должен предшествовать выбору модели, а не наоборот.

1.1 Этапы и уровни анализа

1. **Одномерный анализ (Univariate Analysis):** Исследование распределения отдельных признаков. Позволяет оценить форму распределения (нормальное, скошенное, бимодальное), меры центральной тенденции (среднее арифметическое, медиана, мода) и меры разброса (дисперсия, стандартное отклонение, межквартильный размах IQR). *Дополнение:* Визуализация часто осуществляется с помощью гистограмм (histplot) и ящиков с усами (boxplot).
2. **Двумерный анализ (Bivariate Analysis):** Поиск парных зависимостей между двумя переменными. Используются диаграммы рассеяния (Scatter plot) для числовых признаков и группированные столбчатые диаграммы для категориальных. Для оценки силы связи применяются коэффициенты корреляции: Пирсона (для линейных связей числовых признаков) и Спирмена/Кендалла (для нелинейных или ранговых зависимостей).
3. **Многомерный анализ (Multivariate Analysis):** Исследование сложных, сопряженных эффектов нескольких переменных одновременно. Позволяет увидеть, как изменение одного фактора влияет на целевую переменную при фиксированных значениях других. Реализуется через построение тепловых карт корреляционных матриц (Heatmap) или попарных графиков (pairplot).

1.2 Типы признаков

Качественное понимание типа данных критически важно для выбора правильных методов предобработки и визуализации.

- **Числовые (Quantitative):**

- *Непрерывные*: Величины, способные принимать любые значения на вещественном интервале (например, вес, доход, температура, время).
- *Дискретные*: Счетные величины, выраженные целыми числами (например, число комнат в квартире, количество кликов на сайте, число детей в семье).
- **Категориальные (Qualitative):**
 - *Номинальные*: Категории, не имеющие естественного порядка или иерархии (например, цвет глаз, марка автомобиля, город проживания).
 - *Порядковые (Ordinal)*: Категории с явной иерархией или шкалой сравнения (например, уровень образования, воинское звание, оценка удовлетворенности от 1 до 5).

1.3 Предобработка данных (Data Cleaning & Feature Engineering)

Качество данных напрямую определяет качество модели (принцип "Garbage in, garbage out").

1. Обработка пропусков (Imputation):

- Для числовых шкал: заполнение средним значением (если распределение симметрично) или медианой (если есть выбросы).
- Для категориальных: заполнение модой (наиболее частым значением) или выделение в отдельную категорию "Unknown".
- Продвинутое методы: использование специализированных моделей, таких как KNNImputer (заполнение на основе k ближайших соседей) или IterativeImputer.

2. Борьба с аномалиями (Outliers):

Выявление выбросов через правило трех сигм (3σ) для нормально распределенных данных или через межквартильный интервал (IQR). Значения за пределами диапазона $[Q_1 - 1.5 \cdot IQR, Q_3 + 1.5 \cdot IQR]$ считаются потенциальными выбросами.

3. Трансформация распределения:

При наличии правостороннего или левостороннего скола применяются математические преобразования для приближения распределения к нормальному (что важно для многих статистических тестов и линейных моделей): натуральный логарифм $\ln(x)$, десятичный логарифм $\log_{10}(x)$, извлечение корня $\sqrt{x}$ или более сложное преобразование Бокса-Кокса (Box-Cox).

4. Кодирование категориальных признаков:

- *One-Hot Encoding*: Создание N бинарных dummy-переменных для N уникальных категорий номинального признака. Во избежание ловушки мультиколлинеарности (dummy variable trap) одну колонку часто удаляют (drop='first').
- *Label / Ordinal Encoding*: Присвоение целочисленных рангов (0, 1, 2, ...) порядковым признакам с сохранением их естественной иерархии.

5. Масштабирование и нормализация:

- *MinMax Scaling*: Приведение данных к интервалу $[0, 1]$. Чувствителен к выбросам, так как они сжимают полезный диапазон данных.
- *Standard Scaling (Z-score normalization)*: Сдвиг распределения к нулевому среднему и единичной дисперсии ($\mu = 0, \sigma = 1$). Более устойчив к выбросам и предпочтителен для методов, основанных на расстояниях (KNN, SVM) или градиентном спуске.

Лекция 2 (16.02): Машинное обучение. Обучение с учителем (Supervised Learning)

2.1 Математическая и оптимизационная постановка задачи

Пусть $\mathcal{X}$ – пространство объектов (признаков), а $\mathcal{Y}$ – множество возможных ответов (целевая переменная). Задана неизвестная целевая зависимость $y: \mathcal{X} \rightarrow \mathcal{Y}$.

Нам дана **обучающая выборка (Training Set)**, состоящая из пар «объект–ответ»: $\{(x_1, y_1), (x_2, y_2), \dots, (x_l, y_l)\} \subset \mathcal{X} \times \mathcal{Y}$.

Мы ищем алгоритм $a(x, w)$ из заданного параметрического семейства $\mathcal{A}$, где $w \in \mathcal{W}$ – вектор настраиваемых параметров модели. Обучение модели сводится к решению задачи оптимизации на основе **принципа минимизации эмпирического риска (Empirical Risk Minimization, ERM)**:

$$\frac{1}{l} \sum_{i=1}^l L(a(x_i, w), y_i) \xrightarrow{\min_w}$$

где $L(a(x, w), y)$ – **функция потерь (Loss Function)**, характеризующая штраф за отклонение предсказания алгоритма от истинного ответа y .

2.2 Разделение данных и валидация

Для корректной оценки обобщающей способности (Generalization Ability) и настройки архитектуры вся имеющаяся выборка разбивается на независимые блоки:

1. **Тренировочная выборка (Train Set)**: Занимает 60–80% данных. Используется непосредственно оптимизационным алгоритмом для настройки внутренних параметров (весов w) модели.
2. **Валидационная выборка (Validation Set)**: Применяется инженером/исследователем для подбора оптимальных **гиперпараметров** модели (параметры, задаваемые до обучения, такие как коэффициент регуляризации α , скорость обучения или глубина дерева) и предотвращения переобучения. *Дополнение: На практике часто*

используется кросс-валидация (Cross-Validation) для более надежной оценки.

3. **Тестовая выборка (Test Set):** Составляет 20–40% данных. Служит для финального контроля качества. Модель «видит» тест только один раз, когда обучение и валидация полностью завершены, что гарантирует объективную оценку ее работы на новых данных.

2.3 Фундаментальные проблемы: Недообучение и Переобучение

- **Недообучение (Underfitting):** Модель слишком проста (имеет высокое смещение / high Bias). Ей не хватает емкости или параметров, чтобы уловить закономерности в данных. Метрики качества низкие как на тренировочной, так и на валидационной выборках.
- **Переобучение (Overfitting):** Модель избыточно сложна (имеет высокую дисперсию / high Variance). Она идеально подстраивается под тренировочный шум, буквально «зазубривая» обучающие примеры. Ошибка на тренировке стремится к нулю, но резко возрастает на валидационных и тестовых данных, так как модель не способна к обобщению.

Лекция 3 (02.03): Линейные модели. Задача регрессии

Линейная регрессия – это базовый алгоритм машинного обучения для предсказания непрерывных величин ($y \in \mathbb{R}$).

3.1 Математическая модель

В общем случае множественной линейной регрессии предсказание модели вычисляется как скалярное произведение вектора весов на вектор признаков:

$$\hat{y} = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$$

Если ввести фиктивный признак $x_0 = 1$, свободный коэффициент (смещение / bias) w_0 встраивается в вектор параметров, и модель принимает компактный матричный вид: $\hat{y} = \langle w, x \rangle = w^T x$.

3.2 Цели линейного моделирования

1. **Прогнозирование (Prediction):** Получение точечных предсказаний для новых объектов.
2. **Статистический вывод (Inference):** Оценка характера и силы влияния признаков на целевую переменную. Коэффициенты w_i напрямую

отражают вклад конкретного фактора (размер и направление влияния: положительное или отрицательное).

3.3 Регуляризация (Regularization)

Для борьбы с переобучением и мультиколлинеарностью признаков к стандартной функции потерь (MSE) добавляется регуляризационный штраф за величину весов.

- **Ridge Регрессия (L2-регуляризация):** Штрафует модель за квадрат евклидовой нормы весов. Ограничивает рост коэффициентов, делая модель более устойчивой и гладкой, но не обнуляет их:

$$L_{Ridge} = MSE + \alpha \sum_{j=1}^n w_j^2$$

- **Lasso Регрессия (L1-регуляризация):** Штрафует за сумму абсолютных значений весов. Обладает важным свойством занулять наименее информативные коэффициенты, выполняя встроенный **отбор признаков (Feature Selection)** и создавая разреженные модели:

$$L_{Lasso} = MSE + \alpha \sum_{j=1}^n |w_j|$$

- **ElasticNet:** Комбинированный подход, объединяющий оба типа штрафов с весовыми пропорциями, что полезно при наличии сильно скоррелированных признаков.

3.4 Метрики качества регрессии

- **MAE (Mean Absolute Error):** Средняя абсолютная ошибка. Устойчива к выбросам (робастна), так как штрафует линейно.
- **MSE (Mean Squared Error):** Среднеквадратичная ошибка. Жестко штрафует модель за крупные отклонения из-за возведения разностей в квадрат. Является дифференцируемой, что удобно для оптимизации.
- **RMSE (Root Mean Squared Error):** Корень из MSE. Возвращает метрику к исходной размерности целевой переменной, облегчая интерпретацию.
- **Коэффициент детерминации (R^2):** Доля объясненной моделью дисперсии. Значение $R^2 = 1$ говорит об идеальном предсказании, $R^2 = 0$ означает, что модель предсказывает на уровне константного среднего значения, а отрицательные значения указывают на то, что модель работает хуже, чем простое среднее.

Лекция 4 (16.03): Модели классификации и оценка их эффективности

4.1 Полиномиальная регрессия

Является расширением линейных моделей для аппроксимации нелинейных процессов. Переменные искусственно переводятся в пространство высших степеней (например, из $[x_1, x_2]$ генерируются признаки $[x_1, x_1^2, x_1x_2, x_2^2]$). Степень полинома h на практике редко выбирается больше 3 или 4, так как дальнейшее повышение степени вызывает лавинообразное переобучение модели и экспоненциальный рост числа признаков (проклятие размерности).

4.2 Таксономия моделей классификации

- **Линейные модели:** Логистическая регрессия (Logistic Regression), Линейный дискриминантный анализ (LDA), Машина опорных векторов (SVM) с линейным ядром. Строят линейную разделяющую гиперплоскость.
- **Нелинейные модели:** Метод k-ближайших соседей (k-NN), Наивный байесовский классификатор, Ядерный SVM (Kernel SVM), Деревья решений (Decision Trees), Ансамбли (Random Forest, Gradient Boosting).

4.3 Глубокий анализ метрик классификации

Фундаментом для оценки качества классификации служит **Матрица ошибок (Confusion Matrix)**:

	Предсказано Positive	Предсказано Negative
Реально Positive	True Positive (TP)	False Negative (FN) (Ошибка II рода)
Реально Negative	False Positive (FP) (Ошибка I рода)	True Negative (TN)

- **Ассурасу (Правильность):** Доля верно угаданных классов среди всех объектов. $\text{Assurasy} = \frac{TP+TN}{TP+TN+FP+FN}$. Недостаток: бесполезна при дисбалансе классов.
- **Precision (Точность):** Доля реальных объектов целевого класса среди всех, кого модель пометила этим классом. Минимизирует ложные срабатывания (FP). $\text{Precision} = \frac{TP}{TP+FP}$.
- **Recall (Полнота):** Доля успешно обнаруженных объектов целевого класса из всех реальных объектов этого класса. Минимизирует пропуски (FN). $\text{Recall} = \frac{TP}{TP+FN}$.
- **F1-мера:** Гармоническое среднее точности и полноты, защищающее от манипуляций метриками (например, когда модель просто предсказывает один класс). $F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$.
- **ROC-кривая и ROC-AUC:** График зависимости доли истинно положительных ответов (TPR, он же Recall) от доли ложноположительных ответов (FPR) при варьировании порога бинаризации от 0 до 1. Площадь

под этой кривой (AUC) служит комплексной, пороγο-независимой оценкой разделяющей способности классификатора (AUC = 0.5 – случайное угадывание, AUC = 1.0 – идеальный классификатор).

Лекция 5 (23.03): Неравномерное распределение классов и методы семплирования

При сильном дисбалансе классов (например, 99% нормальных транзакций против 1% мошенничества) стандартные алгоритмы оптимизируют общую Ассурасу, игнорируя редкий класс (миноритарный). Для исправления этой проблемы применяется библиотека `imbalanced-learn`.

5.1 Сабдискретизация (Under-sampling) – уменьшение мажоритарного класса

- **RandomUnderSampler**: Случайное отсечение объектов доминантного класса. Ведет к безвозвратной потере потенциально ценной информации.
- **NearMiss**: Информативный отбор объектов мажоритарного класса. Алгоритм вычисляет расстояния между объектами классов. Например, NearMiss-1 выбирает те объекты мажоритарного класса, среднее расстояние от которых до k ближайших объектов миноритарного класса минимально (сохраняет пограничные примеры).
- **Tomek Links (Связи Томека)**: Пара объектов разного класса i и j называется связью Томека, если они являются ближайшими соседями друг для друга и между ними нет других точек. Удаление мажоритарного объекта из такой пары «очищает» межклассовую границу, делая ее более явной для разделяющей гиперплоскости.

5.2 Передискретизация (Over-sampling) – расширение миноритарного класса

- **RandomOverSampler**: Простые дубликаты редких объектов. Провоцирует жесткое переобучение модели на шумах.
- **SMOTE (Synthetic Minority Over-sampling Technique)**: Для каждого объекта редкого класса находятся его k ближайших соседей из того же класса. На векторе, соединяющем объект с его соседом, случайным образом выбирается точка, и генерируется новый полноценный синтетический объект. Это создает правдоподобные новые данные, а не просто копирует старые.
- **ADASYN**: Развитие идеи SMOTE. Алгоритм генерирует больше синтетических данных в тех пространственных зонах редкого класса, которые сильнее всего окружены (зашумлены) объектами мажоритарного класса, то есть там, где модели труднее всего провести разделение.

5.3 Гибридные (комбинированные) методы

- **SMOTEENN**: Сначала применяется SMOTE для генерации синтетических векторов миноритарного класса, после чего полученное пространство очищается методом ENN (Edited Nearest Neighbors). Объекты, чей класс противоречит большинству их соседей, безжалостно удаляются для устранения шума, внесенного при генерации.
 - **SMOTETomek**: Комбинация SMOTE-генерации с последующим поиском и вырезанием связей Томера вдоль всей пограничной зоны для ее максимального очищения.
-

Лекция 6 (30.03): Машина опорных векторов (SVM) и Деревья решений

6.1 Машина опорных векторов (Support Vector Machines, SVM)

Алгоритм строит разделяющую гиперплоскость в пространстве признаков таким образом, чтобы **максимизировать зазор (margin)** между ближайшими объектами противоположных классов (опорными векторами). Чем шире зазор, тем выше обобщающая способность модели.

- **Функция потерь**: Hinge Loss. В задачах бинарной классификации ($y \in \{-1, +1\}$) SVM минимизирует шарнирную функцию потерь: $L_{Hinge}(y) = \max(0, 1 - y \cdot f(x))$. Если объект лежит глубоко на своей стороне разделения ($y \cdot f(x) \geq 1$), штраф равен нулю. Если объект попадает внутрь зазора или классифицируется неверно, штраф растет линейно.
- **Мягкий зазор (Soft Margin SVM)**: Регулируется управляющим гиперпараметром штрафа C :
 - *Малое C* : Модель слабо штрафует за нарушение зазора. Зазор становится широким, модель игнорирует локальный шум и приобретает высокую обобщающую способность (но может недообучиться).
 - *Большое C* : Модель жестко карает за ошибки. Зазор сужается, алгоритм пытается безошибочно разделить каждую точку на тренировке, скатываясь в переобучение.
- **Ядерный трюк (Kernel Trick)**: Для работы с линейно неразделимыми данными применяется математический перенос объектов в новое высокоразмерное пространство через функции ядра $K(x_i, x_j)$. Это позволяет находить линейные разделяющие плоскости там, где в исходном пространстве граница была сложной нелинейной кривой, при этом вычислительно эффективно (без явного вычисления координат в высоком измерении).
 - *Линейное ядро*: $K(x_i, x_j) = x_i^T x_j$

- *Полиномиальное ядро*: $K(x_i, x_j) = (\gamma x_i^T x_j + r)^d$
- *RBF (Радиально-базисное / Гауссово ядро)*:
 $K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$. Параметр γ регулирует радиус видимости отдельных опорных векторов. Высокие значения γ приводят к локальному переобучению вокруг точек данных.

6.2 Деревья решений (Decision Trees)

Непараметрический алгоритм, строящий логическое дерево условий ветвления для принятия решений.

Эволюционная цепочка алгоритмов:

1. **ID3**: Исторически первый базовый алгоритм. Работает только с дискретными признаками. Выбирает сплиты на основе максимизации прироста информации (Information Gain) через подсчет энтропии Шеннона.
2. **C4.5**: Усовершенствованный вариант ID3. Научился динамически работать с непрерывными числовыми признаками (путем их сортировки и поиска оптимального порога квантования). Вместо Information Gain использует модифицированный критерий **Gain Ratio** для борьбы со смещением в пользу признаков с большим количеством уникальных значений. Реализует процедуру «обрезки» ветвей (Pruning) для борьбы с переобучением.
3. **C5.0**: Проприетарная оптимизация C4.5. Требуется значительно меньше памяти, работает быстрее, поддерживает взвешивание ошибок классов и встроенное автоматическое слияние ветвей (бустинг-подобные структуры).
4. **CART (Classification and Regression Trees)**: Строит строго бинарные деревья (у каждого узла всегда ровно два потомка). Для классификации использует критерий информативности **Джини (Gini Impurity)**, а для регрессии – дисперсию. Одинаково эффективно решает задачи как классификации, так и регрессии. Оптимизированная версия алгоритма CART лежит в основе модуля tree в библиотеке Scikit-learn.

Лекция 7: Нейронные сети и глубокое обучение (Deep Learning)

Нейронные сети (Neural Networks) — это класс алгоритмов машинного обучения, вдохновленный биологической структурой мозга. В отличие от линейных моделей, нейросети являются **универсальными аппроксиматорами**: согласно теореме Цыбенко (Cybenko's Theorem), даже одно скрытый слой с достаточным количеством нейронов и нелинейной функцией активации способен аппроксимировать любую непрерывную функцию с любой заданной точностью.

Глубокое обучение (Deep Learning) — это подобласть машинного обучения, изучающая нейронные сети с большим числом скрытых слоев (обычно более 2-

3). Приставка «глубокое» отражает именно **глубину архитектуры** — количество последовательных преобразований, через которые проходят данные. Именно глубина позволяет сетям автоматически извлекать иерархические признаки: от простых паттернов (границы, углы, текстуры) до сложных семантических концепций (лица, объекты, смыслы).

7.1 Искусственный нейрон (Перцептрон)

Базовый строительный блок любой нейросети — **искусственный нейрон**. Он математически эквивалентен линейной модели с последующей нелинейной трансформацией.

Для входного вектора $\mathbf{x} = (x_1, x_2, \dots, x_n)$ выход нейрона вычисляется по формуле:

$$z = \sum_{i=1}^n w_i x_i + b = \mathbf{w}^T \mathbf{x} + b$$
$$a = \varphi(z)$$

где:

- w_i — **веса (weights)**, определяющие важность каждого входа;
- b — **смещение (bias)**, позволяющее сдвигать функцию активации;
- $\varphi(z)$ — **функция активации**, вносящая нелинейность.

Без функции активации многослойная сеть вырождалась бы в одну линейную модель, так как композиция линейных функций остаётся линейной: $W_2(W_1\mathbf{x} + b_1) + b_2 = W\mathbf{x} + b$.

7.2 Функции активации (Activation Functions)

Функции активации — это сердце нейронных сетей. Они вносят **нелинейность**, без которой невозможна аппроксимация сложных зависимостей. Выбор функции активации напрямую влияет на скорость сходимости и стабильность обучения.

7.2.1 Классические функции

- **Sigmoid (Логистическая функция):**

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Диапазон: $(0, 1)$. *Применение:* Выходной слой бинарной классификации (интерпретируется как вероятность). *Недостатки:* **Проблема затухающих градиентов** (Vanishing Gradient) — при больших $|z|$ производная $\sigma'(z) \approx 0$, и обучение глубоких сетей останавливается. Также выход не центрирован вокруг нуля.

- **Hyperbolic Tangent (Tanh):**

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} = 2\sigma(2z) - 1$$

Диапазон: $(-1, 1)$. *Применение:* Скрытые слои RNN (в LSTM-ячейках). *Преимущество:* Выход центрирован вокруг нуля, что ускоряет сходимость по сравнению с Sigmoid. *Недостаток:* Та же проблема затухающих градиентов.

7.2.2 Современные ReLU-семейства

- **ReLU (Rectified Linear Unit):**

$$\text{ReLU}(z) = \max(0, z)$$

Диапазон: $[0, +\infty)$. *Применение:* Стандарт для скрытых слоёв в CNN и MLP. *Преимущества:* Вычислительно эффективна (простая операция сравнения), решает проблему затухающих градиентов для положительных z , создаёт **разреженные активации** (sparse activations) — часть нейронов «молчит», что ускоряет обучение. *Недостаток:* **Проблема «мёртвых нейронов» (Dying ReLU)** — если нейрон один раз ушёл в отрицательную зону, его градиент навсегда обнулится, и он перестанет обучаться.

- **Leaky ReLU:**

$$\text{LeakyReLU}(z) = \begin{cases} z, & \text{если } z > 0 \\ \alpha z, & \text{если } z \leq 0 \end{cases}$$

где α — малая константа (обычно 0.01). Решает проблему мёртвых нейронов, пропуская маленький градиент в отрицательной области.

- **Parametric ReLU (PReLU):** Параметрическая версия Leaky ReLU, где наклон α в отрицательной зоне — обучаемый параметр сети.
- **GELU (Gaussian Error Linear Unit):**

$$\text{GELU}(z) \approx 0.5z \left(1 + \tanh \left[\sqrt{2/\pi} (z + 0.044715z^3) \right] \right)$$

Применение: Современный стандарт для **Transformer-архитектур** (BERT, GPT, ViT). Более гладкая, чем ReLU, что улучшает сходимость в глубоких сетях.

7.2.3 Выходные функции

- **Softmax:** Обобщение Sigmoid на многоклассовую классификацию. Преобразует вектор логитов $\mathbf{z} \in \mathbb{R}^K$ в распределение вероятностей:

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Сумма выходов всегда равна 1, что позволяет интерпретировать их как вероятности принадлежности к каждому из K классов.

7.3 Архитектура нейронных сетей: типы слоёв

Нейронная сеть — это направленный граф, в котором нейроны организованы в **слои**. Каждый слой выполняет специфическое преобразование данных.

7.3.1 Базовые слои

- **Dense / Fully Connected (Полносвязный слой)**: Каждый нейрон слоя соединён со всеми нейронами предыдущего слоя. Параметров: $n_{in} \cdot n_{out} + n_{out}$. Основа MLP, но плохо масштабируется на изображения из-за взрывного роста числа весов.
- **Input Layer**: Служит точкой входа данных. Определяет размерность входного тензора (например, 28×28 для MNIST или $224 \times 224 \times 3$ для изображений).
- **Output Layer**: Финальный слой, архитектура которого определяется задачей:
 - 1 нейрон + Sigmoid → бинарная классификация;
 - K нейронов + Softmax → многоклассовая классификация;
 - 1 нейрон без активации → регрессия.

7.3.2 Специализированные слои

- **Dropout**: Случайным образом «отключает» (обнуляет) часть нейронов с вероятностью p (обычно 0.2–0.5) на каждом шаге обучения. Это предотвращает ко-адаптацию нейронов и работает как мощный **регуляризатор**, снижая переобучение. На этапе инференса Dropout выключается, а веса масштабируются.
- **Batch Normalization (BatchNorm)**: Нормализует активации каждого мини-батча к нулевому среднему и единичной дисперсии:

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

Затем применяет обучаемое аффинное преобразование $\gamma\hat{x} + \beta$. Позволяет использовать большие скорости обучения, стабилизирует градиенты и также имеет лёгкий регуляризующий эффект.

- **Conv2D (Свёрточный слой)**: Основа **свёрточных нейронных сетей (CNN)**. Применяет набор обучаемых фильтров (ядер) к входному изображению, извлекая пространственные паттерны. Ключевые свойства:
 - **Разделение весов (Weight Sharing)**: Один и тот же фильтр применяется ко всей карте признаков.

- **Локальная связность:** Нейрон связан только с небольшой локальной областью (receptive field).
- **Инвариантность к сдвигу:** Детектор признака работает в любой части изображения.
- **Pooling (Max Pooling / Average Pooling):** Понижает размерность карт признаков, сохраняя наиболее значимые активации. Max Pooling выбирает максимальное значение в окне, обеспечивая пространственную инвариантность.
- **Recurrent Layers (RNN, LSTM, GRU):** Специализированы для обработки **последовательностей** (текст, временные ряды, аудио). Имеют внутреннюю «память» — скрытое состояние $\mathbf{h}_t$, которое передаётся от шага к шагу:

$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t; \mathbf{w})$$

LSTM решает проблему затухающих градиентов классической RNN через механизм **ворот (gates)**: forget gate, input gate, output gate.

- **Attention / Transformer Layers:** Современный стандарт NLP и Computer Vision. Механизм **самовнимания (Self-Attention)** позволяет каждому токenu последовательности напрямую взаимодействовать с любым другим, минуя рекуррентные зависимости:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

7.4 Обучение нейронных сетей

7.4.1 Функции потерь для нейросетей

- **Binary Cross-Entropy (BCE)** — для бинарной классификации:

$$L_{BCE} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

- **Categorical Cross-Entropy (CCE)** — для многоклассовой классификации:

$$L_{CCE} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c})$$

7.4.2 Обратное распространение ошибки (Backpropagation)

Алгоритм обучения нейросетей, основанный на **цепном правиле дифференцирования (Chain Rule)**. Позволяет эффективно вычислить градиент функции потерь по каждому весу сети, распространяя ошибку от выхода ко входу слой за слоем. После вычисления градиентов веса обновляются оптимизатором.

7.4.3 Оптимизаторы

- **SGD (Stochastic Gradient Descent):** Базовый метод. Обновление весов: $w \leftarrow w - \eta \nabla L(w)$, где η — скорость обучения. Часто используется с **моментом (Momentum)** для ускорения сходимости и преодоления локальных минимумов.
- **Adam (Adaptive Moment Estimation):** Современный стандарт. Комбинирует идеи Momentum и RMSProp — поддерживает скользящие средние первого (градиент) и второго (дисперсия градиента) порядков. Для каждого параметра автоматически подбирает индивидуальную скорость обучения.

7.5 Ключевые архитектуры глубокого обучения

Архитектура	Основная идея	Применение
MLP (Multilayer Perceptron)	Полносвязные слои	Табличные данные, простые задачи
CNN (LeNet, ResNet, EfficientNet)	Свёрточные слои + пулинг	Компьютерное зрение, обработка изображений
RNN / LSTM / GRU	Рекуррентные связи	Временные ряды, последовательности
Transformer (BERT, GPT, ViT)	Self-Attention	NLP, генерация текста, современные CV-модели
GAN (Generative Adversarial Networks)	Состязание генератора и дискриминатора	Генерация изображений, стиливой перенос
Autoencoders / VAE	Сжатие и восстановление данных	Понижение размерности, генерация